

3D Gaussian Splatting: Differentiable Rendering with PyTorch

This notebook implements the **Reference Architecture** described in the breakthrough paper *"3D Gaussian Splatting for Real-Time Radiance Field Rendering"* (Kerbl et al., SIGGRAPH 2023).

The Paper: "3D Gaussian Splatting for Real-Time Radiance Field Rendering"

- **Original Paper:** [Kerbl et al. 2023](#)
- **Core Concept:** Scenes are represented by millions of **3D Anisotropic Gaussians** (ellipsoids) that are projected (splatted) onto the screen. Unlike NeRFs which use expensive neural networks, 3DGS uses a fast, differentiable rasterization pipeline.

Main Innovations

1. **3D Gaussians as Primitives:** Geometry is unstructured (like a point cloud), but each point has a covariance matrix Σ (shape/rotation) and opacity α . This allows modeling complex, semi-transparent structures like hair and smoke.
2. **Tile-Based Rasterizer:** The rendering is explicitly designed for GPU parallelism. The screen is divided into tiles, and Gaussians are sorted by depth and alpha-blended efficiently.
3. **Adaptive Density Control:** The model dynamically grows (clones/splits) and prunes Gaussians during training to put more detail where needed.

Comparison with Other Techniques

3DGS vs. NeRF (Neural Radiance Fields)

- **Paradigm:** NeRF is **Implicit** (Coordinate $\rightarrow$ MLP $\rightarrow$ Color/Density). 3DGS is **Explicit** (List of Gaussians $\rightarrow$ Rasterizer $\rightarrow$ Image).
- **Speed:** NeRF is slow to render (sample hundreds of points per ray). 3DGS renders in **Real-Time (100+ FPS)** because it only computes the value of a Gaussian once per pixel it touches, without ray marching steps.
- **Training:** 3DGS trains significantly faster (minutes vs hours) because the explicit representation is easier to optimize than a deep MLP's weights.

3DGS vs. Precomputed Radiance Transfer (PRT)

- **Geometry:** PRT assumes static geometry (usually meshes) and precomputes light transport. 3DGS learns the geometry and appearance simultaneously.
- **View Dependence:** PRT uses Spherical Harmonics (SH) to model environment lighting. 3DGS *also* uses Spherical Harmonics (stored on each Gaussian) to model view-dependent color (specularity), but it does so in a learned, unstructured field rather than on a fixed mesh surface.

3DGS vs. Ray Tracing

- **Physics:** Ray Tracing simulates light paths (bounces, shadows, reflections). 3DGS is a "Splatting" technique (Forward Rendering). It does not natively handle secondary bounces, shadows, or refractions (though approximations exist).
- **Usage:** Ray Tracing is ground-truth quality for synthetic scenes. 3DGS is currently the state-of-the-art for **Inverse Rendering** (reconstructing real scenes from photos).

3DGS vs. Standard 3D Rasterization

- **Primitives:** Standard rasterization uses **Triangles**. 3DGS uses **Gaussians** (Soft blobs).
- **Alpha Blending:** Standard rasterization often struggles with Order-Independent Transparency. 3DGS relies entirely on sorted Alpha Blending, making it excellent for fuzzy objects but requiring a global sort every frame.
- **Differentiability:** Triangle rasterization is hard to differentiate at edges. Gaussian rasterization is smooth and fully differentiable everywhere, which is why we can train it with Gradient Descent.

```
In [ ]: import torch
import torch.nn as nn
import math
import numpy as np
import matplotlib.pyplot as plt
import struct
import os

torch.manual_seed(42)
device = torch.device('cpu') # Use CPU for education/safety
print(f"Using device: {device}")
```

1. The Mathematical Model

1.1 The Anisotropic 3D Gaussian

A 3D Gaussian is defined by: $G(x) = e^{-\frac{1}{2} (x - \mu)^T \Sigma^{-1} (x - \mu)}$ where μ is the center and Σ is the covariance.

1.2 Constructing Covariance (Σ)

Σ must be positive semi-definite. We decompose it into Scaling S and Rotation R : $\Sigma = R S S^T R^T$

1.3 Projecting to 2D (Σ')

To render, we project the 3D covariance to 2D: $\Sigma' = J W \Sigma W^T J^T$ where W is the View Matrix and J is the Jacobian of the projection.

```
In [ ]: def build_rotation(r):
    norm = torch.sqrt(r[:,0]*r[:,0] + r[:,1]*r[:,1] + r[:,2]*r[:,2] + r[:,3]*r[:,3])
    q = r / norm[:, None]
    R = torch.zeros((q.shape[0], 3, 3), device=device)
    r = q[:, 0]; x = q[:, 1]; y = q[:, 2]; z = q[:, 3]
    R[:, 0, 0] = 1 - 2 * (y*y + z*z)
    R[:, 0, 1] = 2 * (x*y - r*z)
    R[:, 0, 2] = 2 * (x*z + r*y)
    R[:, 1, 0] = 2 * (x*y + r*z)
    R[:, 1, 1] = 1 - 2 * (x*x + z*z)
    R[:, 1, 2] = 2 * (y*z - r*x)
    R[:, 2, 0] = 2 * (x*z - r*y)
    R[:, 2, 1] = 2 * (y*z + r*x)
    R[:, 2, 2] = 1 - 2 * (x*x + y*y)
    return R

def build_scaling(s):
    S = torch.zeros((s.shape[0], 3, 3), device=device)
    S[:, 0, 0] = s[:, 0]
    S[:, 1, 1] = s[:, 1]
    S[:, 2, 2] = s[:, 2]
    return S

def compute_covariance_3d(scaling, quaternion):
    R = build_rotation(quaternion)
    S = build_scaling(scaling)
    M = torch.bmm(R, S)
    return torch.bmm(M, M.transpose(1, 2))

def compute_2d_covariance(covariance_3d, view_matrix, world_pos, width, height):
    R_view = view_matrix[:3, :3]
    t_view = view_matrix[:3, 3]
    pos_view = torch.matmul(world_pos, R_view.T) + t_view
    x, y, z = pos_view[:, 0], pos_view[:, 1], pos_view[:, 2]

    focal_x = width / (2.0 * tan_fovx)
    focal_y = height / (2.0 * tan_fovy)
    z = torch.clamp(z, min=0.001)

    J = torch.zeros((world_pos.shape[0], 2, 3), device=device)
    J[:, 0, 0] = focal_x / z
    J[:, 0, 2] = -(focal_x * x) / (z * z)
    J[:, 1, 1] = focal_y / z
```

```
J[:, 1, 2] = -(focal_y * y) / (z * z)

W = R_view.unsqueeze(0).expand(world_pos.shape[0], 3, 3)
T = torch.bmm(J, W)
cov2d = torch.bmm(T, torch.bmm(covariance_3d, T.transpose(1, 2)))
cov2d[:, 0, 0] += 0.3; cov2d[:, 1, 1] += 0.3
return cov2d, pos_view
```

2. Differentiable Rasterization

The core mechanism enables "learning":

1. **Gaussian Solution:** A Gaussian decreases smoothly. Moving it changes pixel opacity smoothly $\rightarrow$ Gradients exist!
2. **Chain Rule:** Error $\rightarrow$ Color $\rightarrow$ Opacity $\rightarrow$ Position. $\frac{\partial \text{Loss}}{\partial \mu} \approx (\text{Target} - \text{Render}) \times \dots$ The system effectively pulls Gaussians towards where they should be to minimize error.

```
In [ ]: def differentiable_rasterizer(world_pos, colors, opacities, scales, rots, width, height):
    fov_y = 60.0
    tan_fovy = math.tan(math.radians(fov_y) * 0.5)
    tan_fovx = tan_fovy * (width / height)
    view_mat = torch.eye(4, device=device)
    view_mat[2, 3] = -5.0
    view_mat = torch.inverse(view_mat)

    cov3d = compute_covariance_3d(scales, rots)
    cov2d, pos_view = compute_2d_covariance(cov3d, view_mat, world_pos, width, height)

    focal_x = width / (2.0 * tan_fovx)
    focal_y = height / (2.0 * tan_fovy)
    screen_x = (pos_view[:, 0] / pos_view[:, 2]) * focal_x + width / 2.0
    screen_y = (pos_view[:, 1] / pos_view[:, 2]) * focal_y + height / 2.0

    depths = pos_view[:, 2]
    sorted_idx = torch.argsort(depths, descending=True)

    y_grid, x_grid = torch.meshgrid(torch.arange(height, device=device), torch.arange(width, device=device))
    canvas = torch.zeros((height, width, 3), device=device)

    for idx in sorted_idx:
        canvas = canvas.clone()

        mu = torch.stack([screen_x[idx], screen_y[idx]])
        sigma = cov2d[idx]
        color = colors[idx]
        opacity = opacities[idx]

        try: inv_sigma = torch.inverse(sigma)
        except RuntimeError: continue

        radius = 3.0 * torch.sqrt(torch.max(torch.abs(sigma)))
        radius = torch.clamp(radius, min=1.0, max=width)
```

```

min_x = int(torch.clamp(mu[0] - radius, 0, width).item())
max_x = int(torch.clamp(mu[0] + radius, 0, width).item())
min_y = int(torch.clamp(mu[1] - radius, 0, height).item())
max_y = int(torch.clamp(mu[1] + radius, 0, height).item())

if max_x <= min_x or max_y <= min_y: continue

grid_x_slice = x_grid[min_y:max_y, min_x:max_x].float()
grid_y_slice = y_grid[min_y:max_y, min_x:max_x].float()
dx = grid_x_slice - mu[0]
dy = grid_y_slice - mu[1]

power = -0.5 * (inv_sigma[0, 0] * dx * dx + 2 * inv_sigma[0, 1] * dx
alpha_splat = torch.exp(power) * opacity
alpha_splat = alpha_splat.unsqueeze(-1)

current_bg = canvas[min_y:max_y, min_x:max_x].clone()
blended_slice = color * alpha_splat + current_bg * (1.0 - alpha_splat)
canvas[min_y:max_y, min_x:max_x] = blended_slice

return canvas

```

```

In [ ]: def optimization_demo():
    print("Starting Optimization Demo...")
    W, H = 64, 64
    target_canvas = torch.zeros((H, W, 3), device=device)
    xv, yv = torch.meshgrid(torch.arange(H), torch.arange(W), indexing='ij')
    dist = torch.sqrt((xv - W/2)**2 + (yv - H/2)**2)
    mask = dist < 15
    target_canvas[mask] = torch.tensor([0.0, 1.0, 0.0]) # Green Circle Target

    # Init 1 Gaussian
    # Init 3 Gaussians to better approximate the circle
    # We initialize them randomly around the center to show "learning"
    num_gaussians = 3
    xyz = nn.Parameter(torch.rand(num_gaussians, 3, device=device) * 2.0 - 1)
    color = nn.Parameter(torch.rand(num_gaussians, 3, device=device))
    # Init scale smallish
    scale = nn.Parameter(torch.tensor([[-2.0, -2.0, -2.0]] * num_gaussians,
    rot = nn.Parameter(torch.rand(num_gaussians, 4, device=device))
    opacity = nn.Parameter(torch.tensor([0.5] * num_gaussians, device=device)

    optimizer = torch.optim.Adam([xyz, color, scale, opacity], lr=0.01)
    losses = []

    for i in range(1001):
        optimizer.zero_grad()
        # Use torch.exp(scale) to prevent scale collapse, and sigmoid for co
        render = differentiable_rasterizer(xyz, torch.sigmoid(color), torch.
        loss = torch.mean(torch.abs(render - target_canvas))
        loss.backward()
        optimizer.step()
        losses.append(loss.item())
        if i % 100 == 0: print(f"Iter {i}: Loss {loss.item():.4f}")

```

```
plt.figure(figsize=(10,3))
plt.subplot(1,3,1); plt.imshow(target_canvas.detach().cpu()); plt.title("Target")
plt.subplot(1,3,2); plt.imshow(render.detach().cpu()); plt.title(f"Learned Render")
plt.subplot(1,3,3); plt.plot(losses); plt.title("Loss")
plt.show()
```

```
optimization_demo()
```

```
In [ ]: def load_ply_tensor(path, max_gaussians=3000):
    print(f"Loading {path}...")
    with open(path, "rb") as f:
        properties = []
        num_verts = 0
        while True:
            line = f.readline().decode("utf-8", errors='ignore').strip()
            if line == "end_header": break
            if line.startswith("element vertex"): num_verts = int(line.split()[1])
            if line.startswith("property"): properties.append(line.split()[1:])

        print(f"Points: {num_verts} (Subsampling to {max_gaussians})")

        dtype = np.dtype([(p, np.float32) for p in properties])
        stride = 4 * len(properties)
        read_count = min(num_verts, max_gaussians)

        data = f.read(stride * read_count)
        vertices = np.frombuffer(data, dtype=dtype, count=read_count)

    def get_attr(names, default_val=0.0):
        for name in names:
            if name in vertices.dtype.names: return torch.from_numpy(vertices[name])
        return torch.full((read_count,), default_val)

    xyz = torch.stack([get_attr(['x']), get_attr(['y']), get_attr(['z'])], dim=0)
    scale = torch.stack([torch.exp(get_attr(['scale_0'], -3.0)), torch.exp(get_attr(['scale_1'], -3.0))], dim=0)
    rot = torch.stack([get_attr(['rot_1']), get_attr(['rot_2']), get_attr(['rot_3'])], dim=0)
    opacity = torch.sigmoid(get_attr(['opacity'])).to(device)

    SH_C0 = 0.28209479177387814
    if 'f_dc_0' in vertices.dtype.names:
        r = 0.5 + SH_C0 * get_attr(['f_dc_0'])
        g = 0.5 + SH_C0 * get_attr(['f_dc_1'])
        b = 0.5 + SH_C0 * get_attr(['f_dc_2'])
    else:
        r = get_attr(['red']) / 255.0
        g = get_attr(['green']) / 255.0
        b = get_attr(['blue']) / 255.0

    rgb = torch.stack([r, g, b], dim=1).clamp(0.0, 1.0).to(device)
    return xyz, rgb, opacity, scale, rot

ply_path = "point_cloud.ply"
if os.path.exists(ply_path):
    try:
        xyz, rgb, op, sc, rot = load_ply_tensor(ply_path)
        center = torch.mean(xyz, dim=0); xyz = xyz - center
```

```
with torch.no_grad():
    img = differentiable_rasterizer(xyz, rgb, op, sc, rot, 256, 256)
    plt.figure(figsize=(5,5)); plt.imshow(img.cpu().numpy()); plt.title(
except Exception as e: print(f"Render failed: {e}")
else:
    print("No point_cloud.ply found.")
```